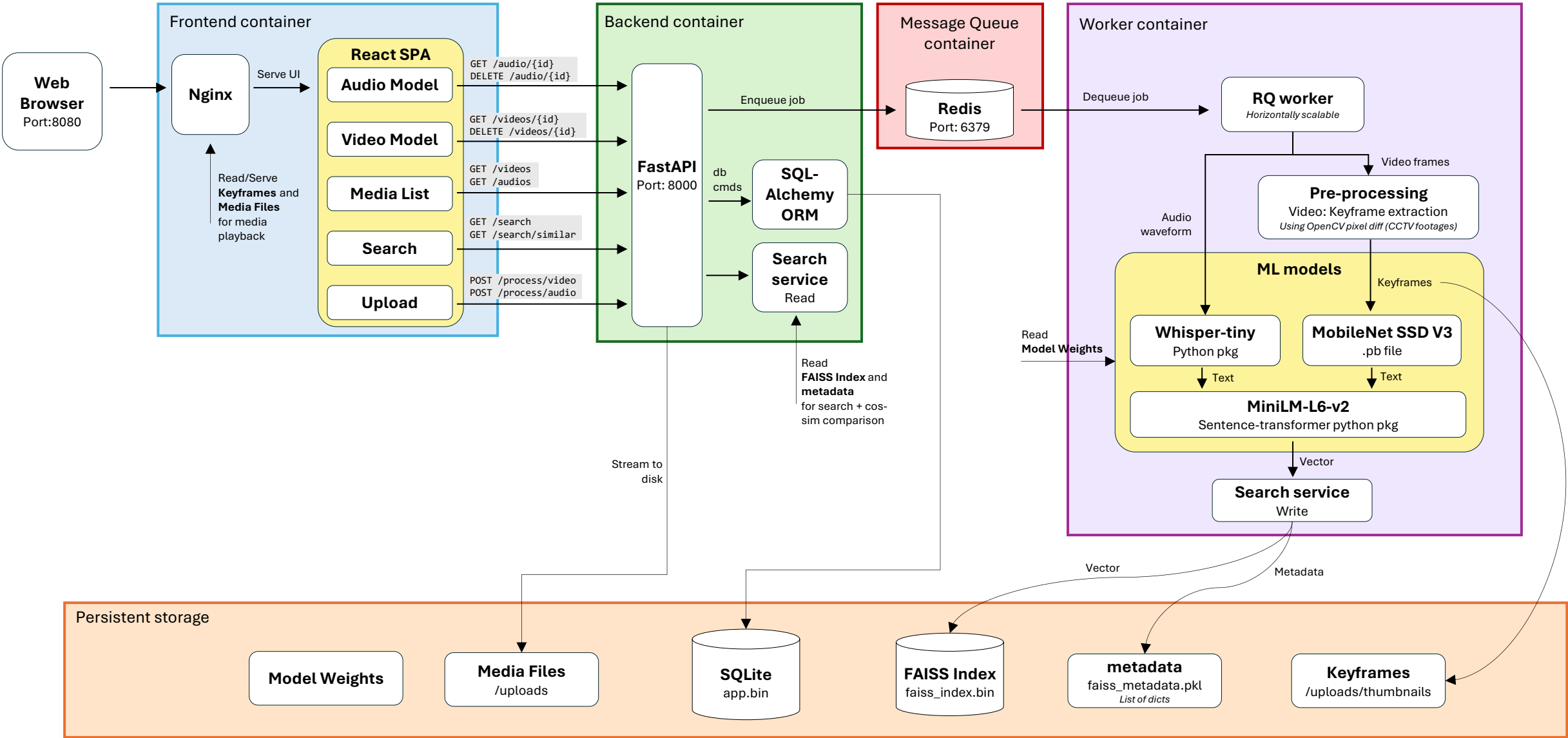
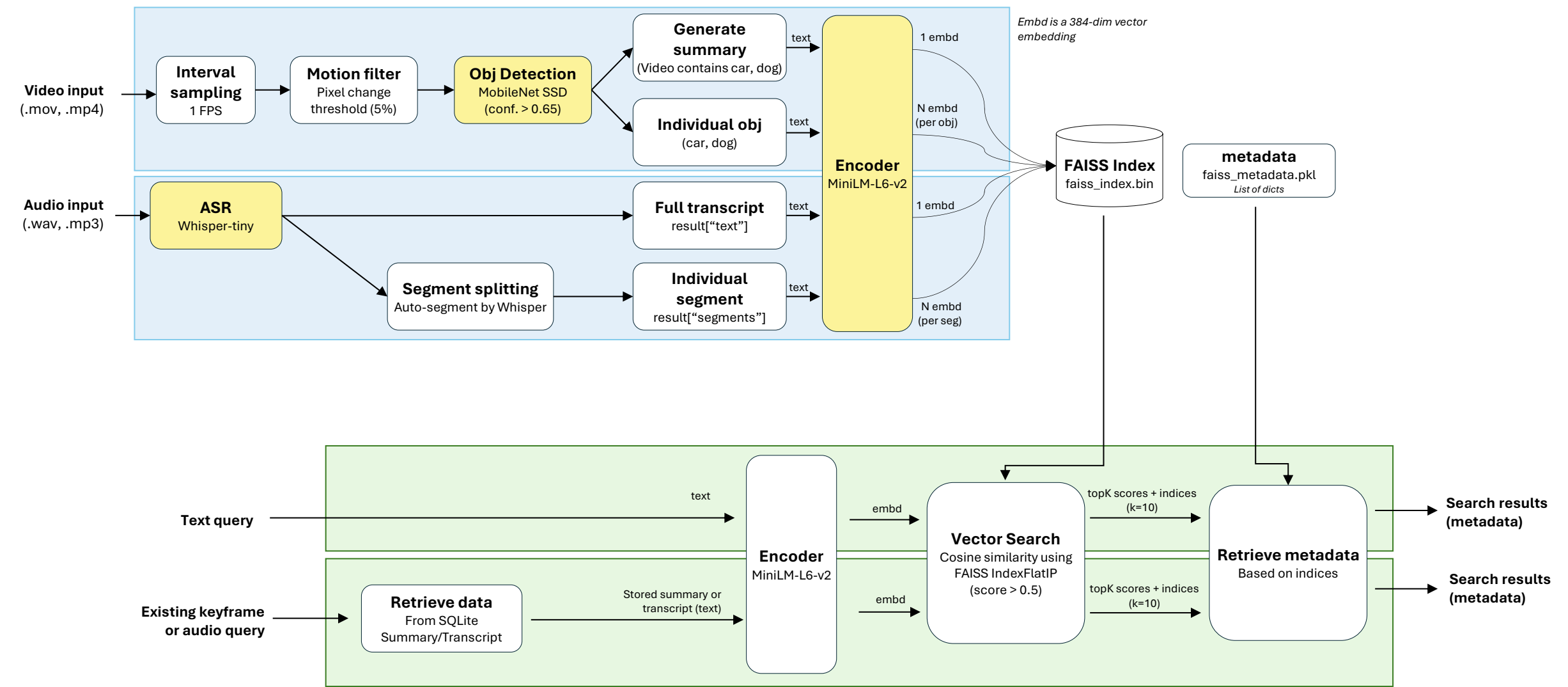


Architecture diagram



Data pipeline

To illustrate (1) Media processing & indexing (2) Text-query search and Content-based similarity search



Backend API endpoints

Swagger file accessible at :8000/docs

Category	Endpoints	Function
System health	GET /health	Monitors system dependencies: SQLite, Redis and FAISS
Media processing	POST /process/video	Uploads video file and enqueues job Creates db record Worker process: extract_keyframes -> detect obj -> summary + obj -> encoder -> FAISS indexing
	POST /process/audio	Upload audio file and enqueues job Creates db record Worker process: transcribe -> segment + full text -> encoder -> FAISS indexing
Content retrieval	GET /videos	Return list of all video entries from SQLite DB, including metadata
	GET /audio	Return list of all audio entries from SQLite DB, including metadata
	GET /videos/{video_id}	Return metadata for specific video Use for video playback + display more information
	GET /audio/{audio_id}	Return metadata for specific audio Use for audio playback + display more information
Search	GET /search?query={text}&type={video audio}	Generates embedding for query text, then search FAISS index for similar vectors Returns topK results with similarity scores. Optional type filter for media type
	GET /search/similar?id={id}&query={text}&type={video audio}	Uses stored summary/transcript of desired item as search query, then search FAISS index for similar vectors Returns topK results with similarity scores. Optional type filter for media type
Deletion	DELETE /videos/{video_id}	Deletes video file, keyframes from system. Deletes metadata from SQLite DB and FAISS index
	DELETE /audio/{audio_id}	Deletes audio file from system. Deletes metadata from SQLite DB and FAISS index

Components consideration

- **Frontend container (Nginx). Nginx serves three critical roles:**
 - **Static Web Server**
 - Serves the compiled React application at port :8080
 - **Media Asset Server**
 - Serves uploaded videos/audio files from /data/uploads/*
 - Serves generated keyframes/thumbnails from /data/uploads/thumbnails/*
 - Uses volume mount: ./backend/data:/usr/share/nginx/html/data
 - Enables React to display media using direct paths (e.g.,)
 - **Single Entry Point**
 - Users only need to access :8080 for the entire UI experience
 - API calls to :8000 are made directly from the browser (CORS enabled on FastAPI)
- **Backend container (FastAPI)**
 - API gateway that handles uploads and manages SQLite DB
 - Enqueues processing jobs to Redis
- **Message queue container (Redis)**
 - Stores job queue using BLPOP (blocking left pop) for efficient dequeue
 - Ensures atomic consumption (each job goes to exactly one worker)
 - Workers poll in FIFO order with automatic retries on failure
- **Worker container**
 - Consumes heavy AI tasks from Redis to prevent blocking the API
 - Can be horizontally scaled: docker-compose up --scale worker=N
 - Each worker loads its own ML models
- **Volumes**
 - /data: Shared storage for uploaded media, SQLite DB (app.db), and FAISS indices
 - /models: Cache for downloaded AI model weights (Whisper/MobileNet)

Design consideration

- **Vector similarity search**
 - **Unified embedding space**
 - All media -> text -> same encoder -> embeddings. This enables cross-media search with one-set of index mapping
 - **Granular indexing**
 - Videos: per object labels. Audio: per segment
 - Able to find exact phrase matches
 - **Similarity metrics** using IndexFlatIP (cosine similarity)
- **Handling media files**
 - **Async processing**
 - Redis queue helps to make sure API returns immediately so UI stays responsive
 - **Video preprocessing**
 - Interval sampling: 1FPS to cut down compute, assuming video is not fast-action
 - Motion filtering: Skip frames with <5% pixel change.
 - Considering sample videos are CCTV footages, using histogram/brightness changes are not suitable since camera is static
 - Streaming save using `shutil.copyfileobj()` during upload
- **Potential improvements (more compute resources given)**
 - **ML model improvements**
 - Whisper (tiny -> large-v3). Better WER
 - MobileNet SSD -> YOLOv8. More classes, better mAP
 - Embeddings (miniLM -> CLIP)
 - **Database improvements**
 - Vector indexing (IndexFlatIP -> IndexIVFPQ). Improves speed from $O(n)$ to $O(\log n)$
 - SQLite to PostgreSQL + pgvector to support native vector operations